

MLIR Part 8 - GPU Compilation with MLIR

Stephen Diehl

GPU Compilation



Continuing on from last time, now that we have our transformer primitives that we need to optimize, let's look at how we can translate high-level tensor operations (like `softmax`, `attention`, etc) expressed in MLIR down to low-level code that can run in parallel on Nvidia GPUs to improve our performance. To do that we're going to start to put together the making of a small compiler.

Now we're going to need to switch to a machine with an Nvidia GPU (sorry no MacOS). So either fire up your Linux machine with a Nvidia GPU or go rent one from the various providers.

[Google Colab](#)

[RunPod](#)

[LambdaLabs](#)

Installing CUDA Toolkit notoriously painful, so if you rent a GPU from a cloud provider this is already done for you. If you need to setup a a bespoke instance you should use one of the official docker images from Nvidia like:

[nvidia/cuda:12.1.0-devel-ubuntu22.04](#)

[nvidia/cuda:12.8.1-cudnn-runtime-ubuntu22.04](#)

```
docker pull nvidia/cuda:12.1.0-devel-ubuntu22.04
```

```
docker run -it --gpus all nvidia/cuda:12.1.0-devel-ubuntu22.04 bash
```

But if you want/need to install it on your own machine here's some instructions, although you should refer to the many guides online for the most up to date instructions. If you have a machine which does not have CUDA Toolkit installed, you can install it using the following steps.

```
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/cuda-keyring_1.0-1_all.deb
sudo dpkg -i cuda-keyring_1.0-1_all.deb
sudo apt-get update
sudo apt-get install build-essential

sudo apt-get -y install cuda-toolkit-12-x
```

Make sure you have the nvcc compiler installed, again if you are renting a GPU this is probably already done for you.

```
$ nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2021 NVIDIA Corporation
Built on Thu_Nov_18_09:45:30_PST_2021
Cuda compilation tools, release 11.5, V11.5.119
Build cuda_11.5.r11.5/compiler.30672275_0
```

Verify that you have a Nvidia GPU installed:

```
lspci | grep -i nvidia
```

And then test the CUDA version and your driver version:

```
$ nvidia-smi
+-----+
+
| NVIDIA-SMI 535.183.01      Driver Version: 535.183.01   CUDA Version: 12.2
|                               |
+-----+-----+
+-----+
```

Note the specific CUDA version (In this case 12.2) this is important because you'll need to instal libraries specific to this version.

Now normally we would have to recompile MLIR with the CUDA runner enabled, which sucks. So I've made a docker image that has MLIR with the CUDA runner enabled. So if you're renting a GPU from RunPod or LambdaLabs you can just use this base image. Or if you have a local machine you can pass the GPU through to the container using [NVIDIA Container Toolkit](#).

```
docker pull ghcr.io/sdiehl/docker-mlir-cuda:main
docker run -it ghcr.io/sdiehl/docker-mlir-cuda:main bash
```

Otherwise you have compile MLIR from source again. Here are the steps:

```
git clone https://github.com/llvm/llvm-project
```

```
mkdir llvm-project/build
cd llvm-project/build
RUN cmake -G Ninja ../llvm \
  -DCUDACXX=/usr/local/cuda/bin/nvcc \
  -DCUDA_PATH=/usr/local/cuda \
  -DCMAKE_CUDA_ARCHITECTURES="75;80;86;90" \ # Add your GPU architecture here
  -DCMAKE_C_COMPILER=clang \
  -DCMAKE_CXX_COMPILER=clang++ \
  -DCMAKE_CUDA_COMPILER=/usr/local/cuda/bin/nvcc \
  -DLLVM_ENABLE_PROJECTS=mlir \
  -DLLVM_BUILD_EXAMPLES=ON \
  -DLLVM_TARGETS_TO_BUILD="Native;NVPTX;AMDGPU" \
  -DCMAKE_BUILD_TYPE=Release \
  -DLLVM_ENABLE_ASSERTIONS=ON \
  -DLLVM_CCACHE_BUILD=ON \
  -DMLIR_ENABLE_CUDA_RUNNER=ON \
  -DMLIR_ENABLE_CUDA_CONVERSIONS=ON \
  -DMLIR_ENABLE_NVPTXCOMPILER=ON

RUN cmake --build . -t mlir-opt mlir-translate mlir-transform-opt mlir-runner
RUN cmake --build . -t install
```

Heads up that this is going to be a somewhat weird crash course in CUDA programming. After all our goal is to build a compiler that targets the GPU from MLIR, so we're going to very quickly cover GPU basics and then delve straight into low-level assembly programming of the GPU very quickly. Most people don't write PTX directly in day to day work, unless they're working on compilers. If this is your first time writing CUDA code, it might be useful to read one of the many guides on CUDA programming online that cover GPU programming in more detail first.

CUDA Ecosystem

The CUDA compilation process involves several key components and intermediate representations that are important to understand for GPU programming. Let's first explore how CUDA programs are compiled and executed.

First CUDA is an extension of C++ that allows you to program the GPU. It's a language built on top of C++ that adds features specifically for GPU programming. It uses its own compiler `nvcc` to compile to PTX (Parallel Thread Execution), which is an intermediate assembly language for NVIDIA GPUs. PTX provides a stable programming model and instruction set for parallel computing. It's architecture-neutral, meaning it can target multiple generations of NVIDIA hardware.

The compilation process then moves from PTX to device-specific binary code called CUBIN (CUDA Binary). A CUBIN file contains the actual machine code that runs on specific GPU architectures. These files are ELF-formatted and contain not just executable code, but also symbols, relocators, and debug information. While CUBIN files are typically embedded in the host executable by default, they can be generated separately using `nvcc's -cubin` option.

nvcc manages this complex compilation process. It coordinates multiple compilation stages and can produce various output formats depending on your needs:

It can generate PTX code for future JIT (Just-In-Time) compilation

It can create architecture-specific CUBIN files

It can combine multiple PTX and CUBIN files into a single FATBIN file

It can embed these GPU binaries into host executables

When you invoke a compiled program on the host it will then do the following:

Determines the available GPU architecture

Selects the appropriate CUBIN from the FATBIN if available

Falls back to JIT-compiling the PTX code if necessary for newer architectures

Loads and executes the code on the GPU

All of this is handled for the end user by the compiler if you use the nvcc compiler.

The FATBIN format is particularly important as it allows a single executable to support multiple GPU architectures. It can contain both PTX code (for future compatibility through JIT compilation) and pre-compiled CUBIN files (for optimal performance on known architectures). Now let's look at some simple C++ code that uses CUDA.

```
#include <stdio.h>

// CUDA kernel function
__global__ void helloKernel() {
    printf("Hello, CUDA!\n");
}

int main() {
    // Call the CUDA kernel
    helloKernel<<<1, 1>>>();

    // Wait for the kernel to finish
    cudaDeviceReset(0);

    return 0;
}
```

In CUDA's extension of C++ there is special syntax for **launching** the kernel. The `helloKernel` function is the kernel we want to execute on the GPU. The `<<<1, 1>>>` is the CUDA syntax for launching the kernel. The first argument is the number of blocks to launch, and the second argument is the number of threads per block.

```
kernel<<<blocks, threads>>>(...)
```

More on threads and blocks later. Now inside of the kernel we have access to special variables `blockDim`,

blockIdx, and threadIdx. As a simple example let's look at a kernel that squares each element of an array. Here we have a square kernel which we'll describe the internals of later, for now it simply squares each element of the array.

```
__global__ void square(int* array, int n) {
    int tid = blockDim.x * blockIdx.x + threadIdx.x;
    if (tid < n)
        array[tid] = array[tid] * array[tid];
}
```

To get a CUBIN code we can use the following command:

```
nvcc -o square square.cu
```

We can run the executable code which contains the compiled CUBIN code in a binary that runs the host logic on the CPU and the CUBIN code on the GPU.

```
./square
```

To get a PTX code out of nvcc we can use the following option:

```
nvcc -ptx square.cu -o square.ptx
```

The PTX code will look like this:

```
.visible .entry square(int*, int)(
    .param .u64 square(int*, int)_param_0,
    .param .u32 square(int*, int)_param_1
)
{

    ld.param.u64    %rd1, [square(int*, int)_param_0];
    ld.param.u32    %r2, [square(int*, int)_param_1];
    mov.u32         %r3, %ntid.x;
    mov.u32         %r4, %ctaid.x;
    mov.u32         %r5, %tid.x;
    mad.lo.s32      %r1, %r3, %r4, %r5;
    setp.ge.s32     %p1, %r1, %r2;
    @%p1 bra        $L__BB0_2;

    cvta.to.global.u64    %rd2, %rd1;
    mul.wide.s32         %rd3, %r1, 4;
    add.s64              %rd4, %rd2, %rd3;
    ld.global.u32        %r6, [%rd4];
    mul.lo.s32           %r7, %r6, %r6;
    st.global.u32        [%rd4], %r7;

$L__BB0_2:
    ret;
```

}

To get an understands of what's going on here, let's break down the PTX instruction by instruction. There are two parameters to the kernel, the first is a pointer to the input integer array `param_0` which is the device address of the input integer array, and the second is `param_1` which is the size of the input integer array.

Instruction	Description
<code>ld.param.u64 %rd1, [square(int*, int)_param_0];</code>	Loads the device address of the input integer array from the kernel parameter.
<code>ld.param.u32 %r2, [square(int*, int)_param_1];</code>	Loads the size of the input integer array from the kernel parameter.
<code>mov.u32 %r3, %ntid.x;</code>	Gets the number of threads per block in the x-dimension.
<code>mov.u32 %r4, %ctaid.x;</code>	Gets the ID of the current thread block in the x-dimension.
<code>mov.u32 %r5, %tid.x;</code>	Gets the ID of the current thread within its block in the x-dimension.
<code>mad.lo.s32 %r1, %r3, %r4, %r5;</code>	Calculates a linear, global-like index for the current thread based on its block and thread ID.
<code>setp.ge.s32 %p1, %r1, %r2;</code>	Sets the predicate flag <code>%p1</code> to true if the calculated index is greater than or equal to the array size (bounds check).
<code>@%p1 bra \$L_BB0_2;</code>	If the calculated index is out of bounds (<code>%p1</code> is true), the thread branches to the return statement, effectively doing nothing.
<code>cvta.to.global.u64 %rd2, %rd1;</code>	Converts the generic device address of the array to a global address.
<code>mul.wide.s32 %rd3, %r1, 4;</code>	Calculates the byte offset into the array for the current thread's element (assuming <code>int</code> size is 4 bytes).
<code>add.s64 %rd4, %rd2, %rd3;</code>	Calculates the absolute global memory address of the array element corresponding to the current thread's index.
<code>ld.global.u32 %r6, [%rd4];</code>	Loads the 32-bit integer value from the calculated global memory address into register <code>%r6</code> .
<code>mul.lo.s32 %r7, %r6, %r6;</code>	Squares the integer value in <code>%r6</code> and stores the lower 32 bits of the result in <code>%r7</code> .
<code>st.global.u32 [%rd4], %r7;</code>	Stores the squared integer value from <code>%r7</code> back into the same global memory location, overwriting the original value.
<code>\$L_BB0_2:</code>	Label marking the point to branch to for out-of-bounds threads.
<code>ret;</code>	Returns from the kernel execution for the current thread.

You can also use the `cuobjdump -sass` command to disassemble the CUBIN code into SASS (SASS stands for "Shader ASSEMBly"):

On `-arch=sm_90` our kernel looks like this:

```
square(int*, int):
LDC R1, c[0x0][0x28]
S2R R5, SR_CTAID.X
ULDC UR4, c[0x0][0x0]
S2R R0, SR_TID.X
```

```

IMAD R5, R5, UR4, R0
ULDC UR4, c[0x0][0x218]
ISETP.GE.AND P0, PT, R5, UR4, PT
@P0 EXIT
LDC.64 R2, c[0x0][0x210]
ULDC.64 UR4, c[0x0][0x208]
IMAD.WIDE R2, R5, 0x4, R2
LDG.E R0, desc[UR4][R2.64]
IMAD R5, R0, R0, RZ
STG.E desc[UR4][R2.64], R5
EXIT
.L_x_0:
BRA `(.L_x_0)
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
.L_x_1:

```

GPU Architectures

The above CUDA kernel, when launched across multiple threads, calculates a unique index for each thread and, if that index is within the bounds of the input integer array, loads the integer at that index from global memory, squares it, and writes the result back to the same memory location, effectively performing an in-place squaring of the array elements in parallel. Threads whose calculated index falls outside the array bounds do not perform any memory operations.

The GPU code such as `sm_53`, consistently begins with the prefix `sm_`. A single virtual GPU architecture may encompass multiple corresponding real architectures. The CUBIN can be executed on all GPUs within the same generation, but it is incompatible with GPUs from older or newer generations. For example, a CUBIN compiled for `sm_52` is compatible with `sm_50`, `sm_52`, and `sm_53` GPUs, but not with `sm_60` GPUs. Furthermore, the performance of a CUBIN compiled for `sm_52` when executed on `sm_53` GPUs may be inferior to that of a CUBIN specifically compiled for `sm_53` running on the same architecture.

```

# Compile for sm_52
nvcc square.cu -o square --gpu-architecture=compute_52 --gpu-code=sm_52

# Compile for sm_52, sm_53, sm_60
nvcc square.cu -o square --gpu-architecture=compute_52 --gpu-
code=sm_52,sm_53,sm_60

```

The most common architectures are:

Architecture	SM Version	GPUs	Features
Maxwell	sm_50, sm_52, sm_53	GTX 750 Ti, GTX 960, GTX 970, GTX 980	Dynamic Parallelism, CUDA
Pascal	sm_60, sm_61, sm_62	P100, GTX 1080, GTX 1080 Ti	Dynamic Parallelism
Volta	sm_70, sm_72	V100, Titan V	HBM2 Memory, NVLink, Unified Memory
Turing	sm_75	T4, RTX 2080, RTX 2080 Ti	First-gen Tensor Cores, Independent Thread Scheduling
Ampere	sm_80, sm_86, sm_87	T4, RTX 2080, RTX 2080 Ti	First-gen RT Cores, Second-gen Tensor Cores
Ada Lovelace	sm_89	A100, A40, RTX 3090	Third-gen Tensor Cores, TF32 precision, Structural Sparsity
Hopper	sm_90, sm_90a	RTX 4090, L40, L40S	Third-gen RT Cores, Fourth-gen Tensor Cores, AV1 encode/decode
Blackwell	sm_100, sm_100a, sm_101, sm_101a, sm_120, sm_120a	H100, H200	Fourth-gen Tensor Cores, Transformer Engine, 900 GB/s NVLink
		B100, B200	Fifth-gen Tensor Cores, Improved occupancy, Enhanced shared memory

Cuda Python Ecosystem

To use NVIDIA CUDA libraries in a Python Poetry project, first add the NVIDIA package index. If you're using Poetry, you can do this by adding the the `nvidia-pyindex` package and following to extra index url.

```
poetry add nvidia-pyindex --source https://pypi.ngc.nvidia.com

[extra]
index-url = "https://pypi.ngc.nvidia.com"
```

The following libraries are now available. The `cu12` suffix indicates the CUDA 12.x version, you may need to adjust this depending on your CUDA version. The one core library you'll need to install is `cuda-python` which is [available on PyPI](#).

`cuda-python` - Pythonic access to CUDA Runtime and other core functionalities

Then the core libraries are:

`nvidia-cuda-runtime-cu12` - Core runtime library providing essential runtime functionality

`nvidia-cublas-cu12` - cuBLAS library for GPU-accelerated Basic Linear Algebra Subroutines

`nvidia-cudnn-cu12` - cuDNN library of deep learning primitives (scaled dot-product attention, convolution, matrix multiplication, softmax, and pooling)

`nvidia-cudnn-frontend` - Python bindings for cuDNN frontend API

There are also instrumentation and compiler utilities:

`nvidia-cuda-nvrtc-cu12` - Runtime Compilation library (NVRTC) for program compilation

`nvml-dev-cu12` - Management Library (NVML) for monitoring and managing GPUs

`nvtx-cu12` - Tools Extension (NVTX) for custom profiling and tracing instrumentation

`cuda-nvcc-cu12` - Compiler (NVCC) for compiling C/C++ code

`nvjitlink-cu12` - JIT Linker for runtime linking of device code

`cuda-sanitizer-api-cu12` - Memory Checker for detecting memory errors in applications

`cuda-cupti-cu12` - Profiling Tools Interface (CUPTI) for performance analysis and profiling

The other libraries are more specialized to specific tasks in scientific computing, signal processing, etc. Not going to cover these in detail here but they exist.

`cuFFT-cu12` - cuFFT library for computing Fast Fourier Transforms

`curand-cu12` - cuRAND library for generating random numbers

`cusolver-cu12` - cuSOLVER library for dense and sparse direct solvers

`cusparse-cu12` - cuSPARSE library for sparse matrix operations

`npp-cu12` - Performance Primitives (NPP) for image, video, and signal processing

`nvjpeg-cu12` - nvJPEG library for hardware-accelerated JPEG encoding/decoding

`opencl-cu12` - Implementation of OpenCL for GPU computing

Threads, Blocks, and Warps

When we write CUDA, a **kernel** refers to a function that you define to execute on the GPU. When you initiate a kernel launch, you are not merely invoking a single function; instead, you are creating hundreds or even thousands of parallel threads that concurrently execute that function across various data elements.

This execution model is commonly known as the Single-Instruction Multiple-Thread (SIMT) paradigm, which emphasizes executing one program across many pieces of data. Say for example we have a kernel that computes the square of each element in an array. Don't worry about the magical `blockDim.x` and `blockIdx.x`, more on that later.

```
__global__ void square(float* array, int n) {  
    int tid = blockDim.x * blockIdx.x + threadIdx.x;  
    if (tid < n)  
        array[tid] = array[tid] * array[tid];  
}
```

CUDA kernels possess two key properties: they cannot explicitly return a value, necessitating that all result data be written to an array passed to the function (for scalar computations, this typically involves passing a one-element array); additionally, kernels must explicitly declare their thread hierarchy upon invocation, specifying the number of thread blocks and the number of threads per block. It is important to note that while a kernel is compiled once, it can be executed multiple times with varying block sizes or grid sizes.

Ok breaking this down, this bit of code is just giving each thread its own unique index, `tid`, to work with in the arrays. The kernel uses that index to let each thread add one element from A and B together. The `__global__`

keyword tells us that this function is a CUDA kernel running on the GPU, and we can kick it off from the CPU side. That if (`tid < n`) check is there because we might end up launching a few more threads than we actually need for the array (we usually round up to a nice number), so any extra threads just sit idle if their index goes out of bounds.

To execute a kernel from our main CPU program, known as the "host code," we must define the number of threads to be used. While the kernel operates on the GPU, it is essential to have CPU code that sets up and launches the kernel. For instance our runner code might look like this:

```
int N = 10000;
int threadsPerBlock = 256;
int numberOfBlocks = (N + threadsPerBlock - 1) / threadsPerBlock;

square<<<numberOfBlocks, threadsPerBlock>>>(d_array, N);
```

To unpack the ideas here, CUDA organizes threads into warps, which consist of 32 threads executing in unison, and these warps are further organized into blocks. Each block operates on a Streaming Multiprocessor (SM), which has finite resources such as registers and shared memory. The size of the block influences the allocation of these resources and determines the number of warps that can run simultaneously, which is known as **occupancy**.

Efficient resource management is crucial for maximizing GPU performance. The GPU scheduler is responsible for distributing blocks across the available SMs. When the number of blocks exceeds the number of SMs, the excess blocks are queued and scheduled for execution as resources become available. This scheduling process is influenced by various factors, including the amount of shared memory and the size of the register file per SM, which ultimately determine the number of blocks that can be executed concurrently.

The other major issue is so-called **warp divergence**. This refers to the situation where threads within a warp diverge in their execution paths due to conditional statements, such as if-statements. Ideally, all threads in a warp should execute the same instruction simultaneously. However, when some threads take one route while others take another, the execution becomes serialized, leading to inefficiencies. The GPU hardware utilizes mask bits to manage which threads should follow each path, ensuring that all threads complete their tasks correctly. While this mechanism preserves correctness, it can adversely affect performance, as idle threads waste computational resources during divergent execution paths. Consequently, the overall throughput of the GPU may decline, particularly in cases with significant divergence. To enhance performance, developers should aim to minimize conditional branching within warps, thereby optimizing GPU execution efficiency.

Memory Hierarchy

There are three types of memory in a GPU:

Register Memory: The fastest type of GPU memory, located directly on the GPU chip. It's accessible only to individual threads and has the shortest lifetime, lasting only for the duration of the thread.

Shared Memory: This memory is shared by all threads within a GPU block. It's faster than global memory but slower than register memory. Shared memory allows for inter-thread communication and data sharing, lasting for the duration of the block.

Global Memory: The largest memory pool in a GPU, accessible by all threads across all blocks. It's slower than

register and shared memory but has the largest capacity. Global memory is used for storing data that needs to be shared across multiple blocks or threads.

The most important performance consideration is **memory transfer overhead** between CPU (host) and GPU (device). Since data transfer between host and device memory is relatively slow (relative to on-chip memory access), it's beneficial to minimize these transfers by keeping data on the GPU as long as possible. This often means performing multiple computations on the same data before transferring results back to the host, rather than transferring data back and forth for each operation. You want to live on the GPU as much as possible only returning to the CPU when necessary.

It's also crucial to understand **memory coalescing** - the ability to combine multiple memory accesses into a single transaction. Memory coalescing occurs when threads in a warp access contiguous memory locations. For optimal performance, developers should ensure that memory access patterns are coalesced whenever possible, as uncoalesced memory access can significantly impact performance by requiring multiple memory transactions.

Memory **bank conflicts** can also impact performance, particularly when using shared memory. Bank conflicts occur when multiple threads attempt to access different addresses in the same memory bank simultaneously. To avoid bank conflicts, you should carefully consider their shared memory access patterns and potentially pad their data structures to ensure optimal memory access distribution across banks. Using appropriate padding and memory access patterns can help maximize memory bandwidth utilization.

Kernel Invocation & Memory Movement

To manage the CUDA runtime there are several core functions [in the Python API](#) (which are just wrappers around the C API). The following are the core functions from the `cuda.cuda` module in the Python bindings.

`cuInit` - Initializes the CUDA driver API.

`cuDeviceGet` - Retrieves a handle to the first compute device.

`cudaSetDevice` - Sets the current device.

`cuCtxCreate` - Creates a compute context on the specified device.

`cuModuleLoadData` - JIT compiles a PTX string to a device binary.

`cuModuleGetFunction` - Retrieves a handle to a kernel function from a module.

`cuLaunchKernel` - Launches a kernel on the specified device.

`cuMemcpyHtoD` - Copies data from host to device memory.

`cuMemcpyDtoH` - Copies data from device to host memory.

`cuMemFree` - Frees memory on the device.

`cuMemAlloc` - Allocates memory on the device.

`cuCtxDestroy` - Destroys a compute context.

`cuModuleUnload` - Unloads a module from the device.

The error handling is a bit annoying in the CUDA Python API because it's a wrapper around C API which uses integer return codes. So for all of these functions you need to check the return value and raise an exception if it's

not a success value. To make this easier there is a helper function `checkCudaErrors` that will raise an exception if the return value is not a success value.

To setup a CUDA context we can use the following boilerplate functions:

```
# gpu_setup.py
# Setup CUDA and create a context.

import cuda.cuda as cu # type: ignore
import cuda.cudart as cudart # type: ignore
import cuda.nvrtc as nvrtc # type: ignore

def _cudaGetErrorEnum(error):
    if isinstance(error, cu.CUresult):
        err, name = cu.cuGetErrorName(error)
        return name if err == cu.CUresult.CUDA_SUCCESS else "<unknown>"
    elif isinstance(error, cudart.cudaError_t):
        return cudart.cudaGetErrorName(error)[1]
    elif isinstance(error, nvrtc.nvrtcResult):
        return nvrtc.nvrtcGetErrorString(error)[1]
    else:
        raise RuntimeError(f"Unknown error type: {error}")

def checkCudaErrors(result):
    if result[0].value:
        raise RuntimeError(
            f"CUDA error code={result[0].value}({_cudaGetErrorEnum(result[0]))}"
        )
    if len(result) == 1:
        return None
    elif len(result) == 2:
        return result[1]
    else:
        return result[1:]

def findCudaDevice():
    devID = 0
    checkCudaErrors(cudart.cudaSetDevice(devID))
    return devID

def findCudaDeviceDRV():
```

```

devID = 0
checkCudaErrors(cu.cuInit(0))
cuDevice = checkCudaErrors(cu.cuDeviceGet(devID))
return cuDevice

def setup_cuda(device_id=None):
    print("Initializing CUDA...")
    # Initialize CUDA
    checkCudaErrors(cu.cuInit(0))

    # Get device
    if device_id is None:
        device = findCudaDeviceDRV()
        device_id = 0 # For printing purposes
    else:
        device = checkCudaErrors(cu.cuDeviceGet(device_id))

    # Create context
    context = checkCudaErrors(cu.cuCtxCreate(0, device))

    print(f"CUDA context created on device {device_id}.")
    return context

def cleanup_cuda(context):
    if context:
        print("Destroying CUDA context...")
        checkCudaErrors(cu.cuCtxDestroy(context))
        print("CUDA context destroyed.")

```

Now let's look at how to manage memory between the host (CPU) and device (GPU) using the CUDA Python API. The following example demonstrates the essential memory operations you'll need for GPU programming:

First, we allocate memory on the GPU using `cuMemAlloc`. This reserves space on the device for our data.

Next, we transfer data from the host to the device with `cuMemcpyHtoD` (Host to Device).

After computations (which we'll cover later), we retrieve results using `cuMemcpyDtoH` (Device to Host).

In our example below, we create a simple float array, send it to the GPU, and then copy it back to verify everything worked correctly. This pattern forms the foundation of GPU computing:

```

# gpu_memory.py
# Minimal example demonstrating CUDA context setup and basic memory movement

from gpu_setup import setup_cuda, checkCudaErrors, cleanup_cuda

```

```

import numpy as np
import ctypes
import cuda.cuda as cu # type: ignore

# Setup CUDA context
context = setup_cuda()

try:
    # Allocate memory on GPU
    buffer_size = 5 * ctypes.sizeof(ctypes.c_float)
    device_ptr = checkCudaErrors(cu.cuMemAlloc(buffer_size))

    # Create and initialize host array
    host_array = np.array([1.0, 2.0, 3.0, 4.0, 5.0], dtype=np.float32)
    print(f"Host array: {host_array}")

    # Copy data from host to device
    checkCudaErrors(cu.cuMemcpyHtoD(device_ptr, host_array.ctypes.data,
buffer_size))

    # Create array for results
    result_array = np.zeros(5, dtype=np.float32)

    # Copy data back from device to host
    checkCudaErrors(cu.cuMemcpyDtoH(result_array.ctypes.data, device_ptr,
buffer_size))
    print(f"Result array: {result_array}")

    # Free GPU memory
    checkCudaErrors(cu.cuMemFree(device_ptr))

finally:
    # Always clean up the context
    cleanup_cuda(context)

```

Now let's launch a simple custom kernel that performs a basic operation. This example shows the complete workflow for executing GPU code directly from PTX assembly without needing to compile CUDA C++ code. The kernel simply sets the first element of an array to the value 42, but it illustrates the essential components of GPU kernel execution: initializing a CUDA context, allocating GPU memory, defining a kernel in PTX assembly language, loading the PTX code into a CUDA module, setting up execution dimensions (a single thread in a single block), launching the kernel with appropriate arguments, synchronizing to ensure completion, and finally transferring the result back to the host for verification. The PTX assembly code includes thread and block ID checks to ensure only the first thread performs the assignment.

```
# gpu_memory.py
```

```

# Minimal example demonstrating launching a kernel

from gpu_setup import setup_cuda, checkCudaErrors, cleanup_cuda
import numpy as np
import cuda.cuda as cu # type: ignore

# This is the trivial kernel that sets the first element
# of the array to 42 using a single thread

# __global__ void set_value_kernel(int *data) {
#     if (threadIdx.x == 0 && blockIdx.x == 0) {
#         *data = 42;
#     }
# }

ptx_kernel = """
.visible .entry kernel(int*)(.param .u64 kernel(int*)_param_0) {
    ld.param.u64    %rd1, [kernel(int*)_param_0];
    mov.u32         %r1, %tid.x;
    mov.u32         %r2, %ctaid.x;
    or.b32          %r3, %r1, %r2;
    setp.ne.s32     %p1, %r3, 0;
    @%p1 bra        $L__BB0_2;
    cvta.to.global.u64 %rd2, %rd1;
    mov.u32         %r4, 42;
    st.global.u32   [%rd2], %r4;
$L__BB0_2:
    ret;
}
"""

# Setup CUDA context
context = setup_cuda()

try:
    # Allocate a single integer on the device
    data_size = np.dtype(np.int32).itemsize
    d_data = checkCudaErrors(cu.cuMemAlloc(data_size))

    # Set up grid and block dimensions - just one thread
    grid_dims = (1, 1, 1)
    block_dims = (1, 1, 1)

```



```

# Prepare arguments for the kernel
args = [d_data]
arg_types = [None] # None for pointer types

# Load the module
module = checkCudaErrors(cu.cuModuleLoadData(ptx_kernel.encode("utf-8")))

# Get kernel function
kernel_func = checkCudaErrors(
    cu.cuModuleGetFunction(module, "kernel".encode("utf-8"))
)

# Prepare kernel arguments
kernel_args = (tuple(args), tuple(arg_types))

# Launch kernel
checkCudaErrors(
    cu.cuLaunchKernel(
        kernel_func,
        grid_dims[0],
        grid_dims[1],
        grid_dims[2],
        block_dims[0],
        block_dims[1],
        block_dims[2],
        0, # shared memory bytes
        0, # stream
        kernel_args, # kernel args
        0, # extra
    )
)

# Synchronize
checkCudaErrors(cu.cuCtxSynchronize())

# Copy result back to host
host_data = np.zeros(1, dtype=np.int32)
checkCudaErrors(cu.cuMemcpyDtoH(host_data.ctypes.data, d_data,
host_data.nbytes))

# Verify result
print(f"Result: {host_data[0]}")
assert host_data[0] == 42, f"Expected 42 but got {host_data[0]}"

```

```

print("Success! Kernel executed correctly.")

# Free device memory
checkCudaErrors(cu.cuMemFree(d_data))

# Unload module
checkCudaErrors(cu.cuModuleUnload(module))

finally:
    cleanup_cuda(context)

```

Thread Blocks and Grids

Previously, we discussed how threads are organized into warps, typically comprising 32 threads each, which are then grouped into blocks. These blocks are assigned to SMs for execution. Now, let's broaden our perspective. A thread block can be understood as a set of threads that work together using shared memory and synchronization mechanisms, all executing on the same SM.

To inspect the GPU we can use the `cuda-python` library to interrogate the GPU driver API to get information about the available parallelism properties.

```

from cuda.cuda import CUdevice_attribute, cuDeviceGetAttribute, cuDeviceGetName,
cuInit

(err,) = cuInit(0)

err, DEVICE_NAME = cuDeviceGetName(128, 0)
DEVICE_NAME = DEVICE_NAME.decode("ascii").replace("\x00", "")

err, MAX_THREADS_PER_BLOCK = cuDeviceGetAttribute(
    CUdevice_attribute.CU_DEVICE_ATTRIBUTE_MAX_THREADS_PER_BLOCK, 0
)

err, MAX_BLOCK_DIM_X = cuDeviceGetAttribute(
    CUdevice_attribute.CU_DEVICE_ATTRIBUTE_MAX_BLOCK_DIM_X, 0
)

err, MAX_GRID_DIM_X = cuDeviceGetAttribute(
    CUdevice_attribute.CU_DEVICE_ATTRIBUTE_MAX_GRID_DIM_X, 0
)

err, SMs = cuDeviceGetAttribute(
    CUdevice_attribute.CU_DEVICE_ATTRIBUTE_MULTIPROCESSOR_COUNT, 0
)

print(f"GPU Device: {DEVICE_NAME}")
print(f"Number of multiprocessors: {SMs}")
print(f"Maximum number of threads per block: {MAX_THREADS_PER_BLOCK:10}")

```

```
print(f"Maximum number of blocks per grid: {MAX_BLOCK_DIM_X:10}")
print(f"Maximum number of threads per grid: {MAX_GRID_DIM_X:10}")
```

For example on an A100 GPU we get the following:

```
Device Name: A100
Maximum number of multiprocessors: 108
Maximum number of threads per multiprocessor: 1024
Maximum number of threads per block: 1024
Maximum number of blocks per grid: 2147483647
```

The use of blocks is driven by practical considerations: there is a hardware limit on the number of threads that can be launched in a single block, typically capped at 1024 threads on modern GPUs. If your computation requires more threads than this limit, you must divide them into multiple blocks. Think of a block as a team of threads collaboratively addressing a specific segment of data. The complete grid represents all the blocks initiated by a kernel, symbolizing the various teams working together to accomplish the overall task.

For example if we wanted to add two arrays of 2048 elements, we could utilize two blocks of 1024 threads each—where block 0 processes indices 0–1023 and block 1 processes indices 1024–2047. In general, if you have N elements and your block can accommodate a maximum of B threads, you would launch $\lceil N/B \rceil$ blocks to ensure all elements are processed.

Blocks are also super handy for scaling and scheduling. Think of it this way: your GPU has a bunch of SMs (let's say 20), and each SM can juggle a few blocks at once depending on what resources are available. So if you throw 100 blocks at it, but only 20 can run at the same time (one on each SM). The GPU kicks off those 20 blocks in parallel, and whenever one wraps up, it immediately grabs another waiting block and puts it to work on that freed-up SM.

From your perspective as a programmer, all 100 blocks are working together to get your answer - it's like having $100 * \text{blockSize}$ threads all running. But behind the scenes, the GPU is cleverly distributing those blocks across its hardware. The beauty is you don't need to stress about launching more threads than your GPU can physically handle at once. The runtime just time-slices those blocks as needed. Plus, blocks give you a natural way to spread work across multiple GPUs or dial back how much parallel work happens at once (which can be a essentiall when you're bumping up against resource limits like shared memory or registers).

Threads within the same block have powerful collaboration capabilities that set them apart. They share access to fast on-chip memory and can synchronize using barrier operations, enabling efficient coordination for tasks requiring inter-thread communication. However, this collaboration is strictly limited to threads in the same block.-Threads across different blocks must resort to slower global memory for communication and cannot directly synchronize with each other. This decision serves an important purpose: it allows for flexible scheduling of blocks across SMs and enables CUDA programs to scale seamlessly across various GPU architectures, regardless of how many SMs they contain.

We typically choose thread block sizes that are powers of 2 (such as 128, 256, or 512) to align with the GPU's warp size of 32 threads and optimize for hardware characteristics. This sizing impacts several performance factors: it maximizes resource utilization, optimizes memory access patterns, and enhances the GPU's ability to effectively hide memory latency through efficient thread scheduling.

```
# N is size of the array to process
```

```
N = 10000
```

```
# We calculate the number of blocks needed based on the total elements and threads
per block
# Typically a power of 2 like 128, 256, or 512
threads_per_block = 256

# Ceiling division to ensure all elements are processed
num_blocks = (N + threads_per_block - 1) // threads_per_block
```

Shared memory in CUDA represents one of the most powerful optimization tools available to improving performance, it functions as a programmable cache that dramatically reduces memory access latency. Located on-chip within each SM, shared memory offers bandwidth that can be 100x higher than global memory while providing latency that's roughly 100x lower. This performance advantage stems from its physical proximity to the compute cores—unlike global memory which resides off-chip and requires data to travel across the much slower PCIe bus or memory controllers. When threads in a block cooperate on calculations involving the same data, shared memory eliminates redundant global memory loads, effectively amortizing the cost of accessing global memory across all threads in the block.

The mechanics behind shared memory reflect CUDA's thread hierarchy design. When a kernel launches, each thread block gets allocated its own dedicated portion of shared memory that persists for the block's lifetime and is visible only to threads within that specific block. This memory is organized into banks (typically 32 banks on modern GPUs) that can be accessed simultaneously by different threads, enabling high-bandwidth parallel access. However, when multiple threads attempt to access different addresses within the same bank (i.e. bank conflicts) these accesses get serialized, degrading performance.

The `__syncthreads()` function is core to working with shared memory. It forces all threads in a block to wait until everyone reaches that point in the code before continuing execution. This prevents nasty race conditions where some threads might try to read data that other threads haven't finished writing yet, which is a common source of subtle and hard-to-debug errors.

In this example we declare a `__shared__` array that is visible to all threads in the block. We then load data from global memory to shared memory, synchronize to make sure all threads have loaded their data, process the data in shared memory, and then write the results back to global memory.

```
#include <stdio.h>

__global__ void sharedMemoryExample(float* input, float* output, int n) {
    // Declare shared memory array - visible to all threads in the block
    __shared__ float sharedData[256];

    // Calculate global thread ID
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    // Load data from global memory to shared memory
    if (tid < n) {
```

```

        sharedData[threadIdx.x] = input[tid];
    }

    // Synchronize to make sure all threads have loaded their data
    __syncthreads();

    // Process data in shared memory (simple example: add 1 to each element)
    if (tid < n) {
        sharedData[threadIdx.x] += 1.0f;
    }

    // Synchronize again before writing results back
    __syncthreads();

    // Write the processed data back to global memory
    if (tid < n) {
        output[tid] = sharedData[threadIdx.x];
    }
}

```

Using shared memory enables algorithmic techniques that would otherwise be impractical. In tiled algorithms, data is processed in small chunks (called tiles) that fit within shared memory, allowing operations like matrix multiplication to achieve near-theoretical peak performance by dramatically reducing global memory traffic. For stencil computations, shared memory allows threads to collaborate on overlapping regions, eliminating duplicate global memory loads. In reduction operations like sum or max calculations, shared memory facilitates efficient parallel reduction patterns where threads progressively combine results within a block. The key design pattern is to load data from global to shared memory, perform computation exclusively on shared memory, then write results back to global memory. This transforms what would otherwise be memory-bound computations into compute-bound ones, effectively leveraging the GPU's massive computational throughput while minimizing its primary bottleneck: memory access latency.

Connecting to MLIR

Ok, so our goal now is to be able to dynamically (i.e. at runtime) specify high-level tensor operations and have them be lowered to the appropriate GPU code using MLIR. We'll leverage MLIR's progressive lowering capabilities to transform high-level tensor operations into optimized GPU code through a series of well-defined stages:

High-Level Representation: We start with abstract tensor computations expressed in the `linalg` dialect (matrix multiplication, convolutions, element-wise operations). These operations represent pure mathematical intent without implementation details, keeping our code close to the domain.

Affine Transformation: Using passes like `-convert-linalg-to-loops` or `-convert-linalg-to-affine-loops`, we transform these high-level operations into the `affine` dialect, which represents loops and memory accesses with precise mathematical relationships. This exposes opportunities for crucial optimizations like tiling, fusion, and loop interchange that improve memory locality and computational efficiency.

GPU Mapping: We then map these loop structures to the GPU's execution model through passes like `-convert-affine-for-to-gpu` and `-convert-parallel-loops-to-gpu`. This transforms our code into the `gpu` dialect, which explicitly represents concepts like thread blocks and threads while remaining hardware-agnostic.

Hardware-Specific Lowering: Finally, we transition to NVIDIA-specific functionality through the `nvvm` dialect, and ultimately to LLVM IR with GPU intrinsics. The final stages convert this to NVPTX assembly and then to CUBIN machine code that runs directly on the GPU.

Throughout this pipeline, each lowering step preserves program semantics while moving closer to the target hardware. We systematically transform from mathematical expressions to explicit loop nests, then to GPU execution constructs, and finally to hardware-specific code. This approach allows us to express computations at a high level while still generating highly optimized GPU code that leverages the full capabilities of the hardware.

The `gpu` Dialect

The GPU dialect in MLIR provides middle-level abstractions for launching GPU kernels that follow programming models similar to CUDA or OpenCL. This dialect abstracts away device-specific and driver-specific operations needed to launch GPU kernels, providing a streamlined path toward GPU execution from MLIR.

The dialect uses `gpu` as its canonical prefix and exposes operations that wrap common GPU primitives. In some idealized future world this might be used to target multiple GPU backends, but for now we'll just focus on the Nvidia specific operations with the goal of lowering to PTX.

Device Operations

`gpu.launch` - Launches a GPU kernel with the specified grid and block dimensions. Takes block and thread configurations and contains the kernel body.

`gpu.launch_func` - Launches a GPU function with the specified grid and block dimensions. Takes a function reference and block and thread configurations.

`gpu.barrier` - Synchronizes all threads within a thread block, ensuring all threads reach this point before continuing execution.

`gpu.binary` - Represents a binary blob containing compiled GPU code (e.g. PTX or CUBIN format) that can be loaded and executed.

`gpu.printf` - Prints formatted output from within GPU kernels, similar to C's `printf` function. Useful for debugging GPU code.

`gpu.return` - Returns values from a GPU function/kernel back to the host.

`gpu.terminator` - Marks the end of a GPU launch region, required as the last operation in a `gpu.launch` body.

`gpu.wait` - Waits for asynchronous GPU operations to complete before continuing host execution.

`gpu.yield` - Yields values from a structured control flow region in GPU code, similar to `scf.yield`.

Thread and Block Operations

`gpu.block_id` - Returns the ID of the current block within the grid along a specified dimension (x, y, or z)

`gpu.block_dim` - Returns the dimensions of blocks in the grid along a specified dimension (x, y, or z)

`gpu.block_size` - Returns the total number of threads in a block along a specified dimension (x, y, or z)

`gpu.grid_id` - Returns the ID of the current grid being executed along a specified dimension (x, y, or z)

`gpu.grid_dim` - Returns the dimensions of the grid along a specified dimension (x, y, or z)

`gpu.grid_size` - Returns the total number of blocks in the grid along a specified dimension (x, y, or z)

`gpu.thread_id` - Returns the ID of the current thread within its block along a specified dimension (x, y, or z)

Memory Operations

`gpu.alloc` - Allocates memory on the GPU device. Takes a shape and element type and returns a memref pointing to the allocated memory.

`gpu.memcpy` - Copies memory between host and device or between different locations on the device. Can handle host-to-device, device-to-host, and device-to-device transfers.

`gpu.dealloc` - Deallocates memory previously allocated on the GPU device using `gpu.alloc`.

`gpu.host_register` - Registers a memref as host-local memory, allowing it to be accessed directly by the host.

`gpu.host_unregister` - Unregisters a memref for access from device.

The dialect is designed to be a target-agnostic representation that can be lowered to specific GPU backends. It provides abstractions for kernel invocations and may eventually include device management capabilities that aren't present at lower levels like LLVM IR intrinsics for GPUs.

The dialect expects GPU code to be organized within `gpu.module` operations and kernels to be represented as `gpu.func` operations. This structure provides a clear separation between host and device code, making it easier to handle the compilation and execution pipeline. Non-kernel functions, such as device library calls, can be defined using `func.func` or other non-GPU dialect operations, providing some flexibility in how GPU programs are structured.

```
module {

  // GPU module
  gpu.module @gpu_module {
    // GPU function
    gpu.func @kernel() {
      %0 = arith.addi %arg0, %arg0 : i32
      %1 = arith.addi %arg0, %arg0 : i32
      %2 = arith.addi %0, %1 : i32
      gpu.return
    }
  }

  // Host function
  func.func @main() {
```



```

    gpu.launch_func @gpu_module::@kernel
      blocks(%0, %1, %2) in (%3 = %c1, %4 = %c1, %5 = %c1)
      threads(%6, %7, %8) in (%9 = %c2, %10 = %c1, %11 = %c1)
      args()
  }
}

```

The alternative form is to use the `gpu.launch` operation which embeds the kernel function in a region passed as an argument to the launch operation. The `gpu.terminator` operation is used to mark the end of the region.

```

func.func @main() {
  %c2 = arith.constant 2 : index
  %c1 = arith.constant 1 : index
  gpu.launch
    blocks(%0, %1, %2) in (%3 = %c1, %4 = %c1, %5 = %c1)
    threads(%6, %7, %8) in (%9 = %c2, %10 = %c1, %11 = %c1) {
      gpu.printf "Hello from %d\n" %6 : index
      gpu.terminator
    }
  return
}

```

nvgpu Dialect

The `nvgpu` dialect functions as an intermediary in the MLIR ecosystem, connecting higher-level target-agnostic dialects like `gpu` and `vector` with the lower-level NVVM dialect specifically designed for NVIDIA GPUs. By representing PTX-specific operations while continuing to use MLIR's high-level abstractions such as `memref` and `tensor` dialects, it creates a bridge that preserves the benefits of both worlds.

Using this dialect we can access NVIDIA-specific hardware features through the NVGPU dialect without needing to directly manipulate complex NVVM intrinsics. This intermediate layer significantly simplifies the generation of efficient GPU code while maintaining the clarity and expressiveness of higher-level MLIR representations.

The dialect primarily focuses on exposing advanced NVIDIA GPU capabilities such as the **Tensor Memory Accelerator** (TMA) for efficient tensor transfers between global and shared memory. It also provides mechanisms for **asynchronous memory operations** that enable computation and memory transfers to overlap, effectively hiding latency. Memory access patterns are optimized through **swizzling** techniques that reduce bank conflicts, while memory barriers (`mbarriers`) offer sophisticated synchronization tools for coordinating operations across threads.

Warp-level programming receives special attention in the NVGPU dialect, with fine-grained controls that expose hardware-specific features like warp matrix multiply-accumulate operations and synchronization primitives. These capabilities are essential for implementing high-performance matrix operations and other compute-intensive workloads by leveraging specialized hardware units on newer NVIDIA GPUs like Hopper and Blackwell chips.

The dialect's position in the compilation pipeline is strategically important, sitting between the more general GPU dialect and the hardware-specific NVVM dialect. This placement allows it to translate higher-level concepts

into optimized hardware-specific operations while shielding developers from unnecessary complexity. When it's time to generate final code, the NVGPU dialect can be lowered to NVVM using the `-convert-nvgpu-to-nvvm` pass, which transforms the NVGPU operations into corresponding NVVM dialect intrinsics.

For now we're going punt on these features, but later on we'll explore these capabilities (particularly TMA and async memory operations) in greater detail.

nvvm Dialect

The NVVM dialect is a target-specific dialect that represents the LLVM IR for NVIDIA GPUs. It includes operations for GPU-specific constructs, such as thread and block indexing, synchronization, and memory operations.

NVVM IR is a compiler IR (intermediate representation) based on the LLVM IR. The NVVM IR is designed to represent GPU compute kernels (for example, CUDA kernels). High-level language front-ends, like the CUDA C compiler front-end, can generate NVVM IR. The NVVM compiler (which is based on LLVM) generates PTX code from NVVM IR.

CUDA Builtin	NVVM Intrinsic	MLIR Operation
<code>threadId.{x,y,z}</code>	<code>@llvm.nvvm.read.ptx.sreg.tid.{x,y,z}</code>	<code>gpu.thread_id {x,y,z}</code>
<code>blockIdx.{x,y,z}</code>	<code>@llvm.nvvm.read.ptx.sreg.ctaid.{x,y,z}</code>	<code>gpu.block_id {x,y,z}</code>
<code>blockDim.{x,y,z}</code>	<code>@llvm.nvvm.read.ptx.sreg.ntid.{x,y,z}</code>	<code>gpu.block_dim {x,y,z}</code>
<code>gridDim.{x,y,z}</code>	<code>@llvm.nvvm.read.ptx.sreg.nctaid.{x,y,z}</code>	<code>gpu.grid_dim {x,y,z}</code>
<code>__syncthreads()</code>	<code>@llvm.nvvm.barrier0()</code>	<code>gpu.barrier</code>

MLIR Python Bindings

The [MLIR Python bindings](#) provide a interface to the MLIR C++ internals, allowing developers to programmatically manipulate MLIR's intermediate representations without relying on command-line tools. These bindings expose the core MLIR functionality through Python, enabling users to parse MLIR text, construct and modify the MLIR AST, apply transformation passes, and even JIT-compile and execute MLIR code directly from Python.

Working with the Python bindings eliminates many of the friction points associated with the traditional command-line approach to MLIR. Rather than chaining together multiple command-line invocations, developers can create end-to-end compilation pipelines entirely within Python, making the development process more interactive and debuggable. However, building these bindings from source can be challenging due to their complex dependencies on LLVM and MLIR. To simplify adoption, pre-built wheels are available that package the bindings with all necessary dependencies, making installation as straightforward as a pip command.

To install the bindings, you can use pip:

```
pip install mlir_python_bindings -f https://github.com/makslevental/mlir-wheels/releases/expanded_assets/latest
```

Or if you're using Poetry add the following to your `pyproject.toml`:

```
[tool.poetry.dependencies]
python = "^3.10"
```

```
mlir-python-bindings = { version = "*", source = "mlir-wheels"}
```

```
[[tool.poetry.source]]
```

```
name = "mlir-wheels"
```

```
url = "https://github.com/makslevental/mlir-wheels/releases/expanded_assets/  
latest"
```

```
priority = "supplemental"
```

Ok now let's use the Python bindings to lower a simple vector addition kernel to PTX.

```
module {  
  func.func @square(%input: tensor<10x10xf32>, %output: tensor<10x10xf32>) ->  
  tensor<10x10xf32> {  
    %x0 = linalg.square ins(%input : tensor<10x10xf32>) outs(%output :  
  tensor<10x10xf32>) -> tensor<10x10xf32>  
    return %x0 : tensor<10x10xf32>  
  }  
}
```

Here we'll apply the passes

```
from mlir.ir import Context, Module  
from mlir.passmanager import PassManager
```

```
mlir_module_str = open("vecadd.mlir").read()
```

```
with Context():
```

```
    # Parse the input module
```

```
    module = Module.parse(mlir_module_str)
```

```
pm = PassManager()
```

```
pm.enable_ir_printing(print_after_change=True)
```

```
pm.add("canonicalize")
```

```
pm.add(  
    "one-shot-bufferize{ bufferize-function-boundaries function-boundary-type-  
conversion=identity-layout-map }"
```

```
)  
pm.add("canonicalize")  
pm.add("convert-linalg-to-affine-loops")  
pm.add("func.func(affine-loop-invariant-code-motion)")  
pm.add("func.func(convert-affine-for-to-gpu)")  
pm.add("gpu-kernel-outlining")  
pm.add("lower-affine")  
pm.add("gpu-decompose-memrefs")  
pm.add("expand-strided-metadata")  
pm.add("normalize-memrefs")
```

```
pm.add(
    "gpu.module(convert-gpu-to-nvvm{index-bitwidth=0 use-bare-ptr-memref-call-conv
})"
)
pm.add(f"nvvm-attach-target{ {chip={chip_type} features=+ptx80 0=3} }")
pm.add("convert-nvvm-to-llvm")
pm.add("reconcile-unrealized-casts")
pm.add("gpu-to-llvm { use-bare-pointers-for-host use-bare-pointers-for-kernels }")
pm.run(module.operation)

print(module)
```

Ok so there's a lot going on here, let's break down each flag in the command:

`one-shot-bufferize{ bufferize-function-boundaries function-boundary-type-conversion=identity-layout-map }`: Converts tensor operations to buffer operations in a single pass, handling function boundaries with identity layout mapping.

`convert-linalg-to-affine-loops`: Transforms high-level linalg operations into affine loop nests that explicitly iterate over tensor elements.

`func.func(affine-loop-invariant-code-motion)`: Performs loop-invariant code motion on affine loops, moving computations outside loops when possible to reduce redundant calculations.

`func.func(convert-affine-for-to-gpu)`: Maps affine loops to GPU execution model, distributing iterations across GPU threads and blocks.

`gpu-kernel-outlining`: Extracts GPU kernel regions into separate GPU functions that can be launched from host code.

`lower-affine`: Converts affine dialect operations to standard control flow and arithmetic operations.

`gpu-decompose-memrefs`: Decomposes complex memref types into simpler ones that can be handled by the GPU backends.

`expand-strided-metadata`: Expands metadata for strided memory accesses to explicit calculations.

`normalize-memrefs`: Normalizes memory references to a form expected by the GPU backends.

`gpu.module(convert-gpu-to-nvvm{index-bitwidth=0 use-bare-ptr-memref-call-conv })`: Converts GPU dialect operations to NVVM dialect (NVIDIA's LLVM-based IR), using bare pointer calling conventions for memrefs.

`nvvm-attach-target{chip={chip_type} features=+ptx80 0=3}`: Attaches target-specific information to the NVVM module, specifying the GPU architecture, PTX version 8.0 features, and optimization level 3.

`convert-nvvm-to-llvm`: Translates NVVM dialect to standard LLVM dialect for further processing.

`reconcile-unrealized-casts`: Resolves any remaining type conversion issues by reconciling unrealized cast operations.

`gpu-to-llvm { use-bare-pointers-for-host use-bare-pointers-for-kernels }`: Converts remaining GPU dialect operations to LLVM dialect, using bare pointers for both host and device code.

After lowering our linalg operations to affine loops with `convert-linalg-to-affine-loops` we get the following MLIR module:

```
module {
  func.func @square(%arg0: memref<10x10xf32>, %arg1: memref<10x10xf32>) ->
    memref<10x10xf32> {
    affine.for %arg2 = 0 to 10 {
      affine.for %arg3 = 0 to 10 {
        %0 = affine.load %arg0[%arg2, %arg3] : memref<10x10xf32>
        %1 = arith.mulf %0, %0 : f32
        affine.store %1, %arg1[%arg2, %arg3] : memref<10x10xf32>
      }
    }
    return %arg1 : memref<10x10xf32>
  }
}
```

After the `convert-affine-for-to-gpu` pass. This pass transforms affine loops into GPU kernel launch region.

```
func.func @square(%arg0: memref<10x10xf32>, %arg1: memref<10x10xf32>) ->
  memref<10x10xf32> {
    %c0 = arith.constant 0 : index
    %c10 = arith.constant 10 : index
    %0 = arith.subi %c10, %c0 : index
    %c1 = arith.constant 1 : index
    %c0_0 = arith.constant 0 : index
    %c10_1 = arith.constant 10 : index
    %1 = arith.subi %c10_1, %c0_0 : index
    %c1_2 = arith.constant 1 : index
    %c1_3 = arith.constant 1 : index
    gpu.launch
      blocks(%arg2, %arg3, %arg4) in (%arg8 = %0, %arg9 = %c1_3, %arg10 = %c1_3)
      threads(%arg5, %arg6, %arg7) in (%arg11 = %1, %arg12 = %c1_3, %arg13 = %c1_3)
    {
      %2 = arith.addi %c0, %arg2 : index
      %3 = arith.addi %c0_0, %arg5 : index
      %4 = affine.load %arg0[%2, %3] : memref<10x10xf32>
      %5 = arith.mulf %4, %4 : f32
      affine.store %5, %arg1[%2, %3] : memref<10x10xf32>
      gpu.terminator
    }
    return %arg1 : memref<10x10xf32>
  }
```

Applying the gpu-kernel-outlining pass. This pass extracts the GPU kernel launch region into a separate GPU function which has the GPU intrinsics to reference the block and thread indices.

```
module attributes {gpu.container_module} {
  func.func @square(%arg0: memref<10x10xf32>, %arg1: memref<10x10xf32>) ->
memref<10x10xf32> {
    %c0 = arith.constant 0 : index
    %c10 = arith.constant 10 : index
    %0 = arith.subi %c10, %c0 : index
    %c1 = arith.constant 1 : index
    %c0_0 = arith.constant 0 : index
    %c10_1 = arith.constant 10 : index
    %1 = arith.subi %c10_1, %c0_0 : index
    %c1_2 = arith.constant 1 : index
    %c1_3 = arith.constant 1 : index
    gpu.launch_func @square_kernel::@square_kernel
      blocks in (%0, %c1_3, %c1_3)
      threads in (%1, %c1_3, %c1_3)
      args(%c0 : index, %c0_0 : index, %arg0 : memref<10x10xf32>, %arg1 :
memref<10x10xf32>)
    return %arg1 : memref<10x10xf32>
  }
  gpu.module @square_kernel {
    gpu.func @square_kernel(%arg0: index, %arg1: index, %arg2: memref<10x10xf32>,
%arg3: memref<10x10xf32>) kernel {
      %block_id_x = gpu.block_id x
      %block_id_y = gpu.block_id y
      %block_id_z = gpu.block_id z
      %thread_id_x = gpu.thread_id x
      %thread_id_y = gpu.thread_id y
      %thread_id_z = gpu.thread_id z
      %grid_dim_x = gpu.grid_dim x
      %grid_dim_y = gpu.grid_dim y
      %grid_dim_z = gpu.grid_dim z
      %block_dim_x = gpu.block_dim x
      %block_dim_y = gpu.block_dim y
      %block_dim_z = gpu.block_dim z
      %0 = arith.addi %arg0, %block_id_x : index
      %1 = arith.addi %arg1, %thread_id_x : index
      %2 = affine.load %arg2[%0, %1] : memref<10x10xf32>
      %3 = arith.mulf %2, %2 : f32
      affine.store %3, %arg3[%0, %1] : memref<10x10xf32>
      gpu.return
    }
  }
}
```

```
}  
}  
}
```

The rest of the passes are fairly mechanical lowering passes that convert the GPU dialect to NVVM dialect. We then get a module with a `gpu.module` that contains the GPU kernel.

```
module attributes {gpu.container_module} {  
  llvm.func @square(%arg0: !llvm.ptr, %arg1: !llvm.ptr) -> !llvm.ptr {  
    %0 = llvm.mlir.poison : !llvm.struct<(ptr, ptr, i64, array<2 x i64>, array<2 x  
i64>)>  
    %1 = llvm.insertvalue %arg1, %0[0] : !llvm.struct<(ptr, ptr, i64, array<2 x  
i64>, array<2 x i64>)>  
    %2 = llvm.insertvalue %arg1, %1[1] : !llvm.struct<(ptr, ptr, i64, array<2 x  
i64>, array<2 x i64>)>  
    %3 = llvm.mlir.constant(0 : index) : i64  
    %4 = llvm.insertvalue %3, %2[2] : !llvm.struct<(ptr, ptr, i64, array<2 x i64>,  
array<2 x i64>)>  
    %5 = llvm.mlir.constant(10 : index) : i64  
    %6 = llvm.insertvalue %5, %4[3, 0] : !llvm.struct<(ptr, ptr, i64, array<2 x  
i64>, array<2 x i64>)>  
    %7 = llvm.mlir.constant(10 : index) : i64  
    %8 = llvm.insertvalue %7, %6[4, 0] : !llvm.struct<(ptr, ptr, i64, array<2 x  
i64>, array<2 x i64>)>  
    %9 = llvm.mlir.constant(10 : index) : i64  
    %10 = llvm.insertvalue %9, %8[3, 1] : !llvm.struct<(ptr, ptr, i64, array<2 x  
i64>, array<2 x i64>)>  
    %11 = llvm.mlir.constant(1 : index) : i64  
    %12 = llvm.insertvalue %11, %10[4, 1] : !llvm.struct<(ptr, ptr, i64, array<2 x  
i64>, array<2 x i64>)>  
    %13 = llvm.mlir.poison : !llvm.struct<(ptr, ptr, i64, array<2 x i64>, array<2  
x i64>)>  
    %14 = llvm.insertvalue %arg0, %13[0] : !llvm.struct<(ptr, ptr, i64, array<2 x  
i64>, array<2 x i64>)>  
    %15 = llvm.insertvalue %arg0, %14[1] : !llvm.struct<(ptr, ptr, i64, array<2 x  
i64>, array<2 x i64>)>  
    %16 = llvm.mlir.constant(0 : index) : i64  
    %17 = llvm.insertvalue %16, %15[2] : !llvm.struct<(ptr, ptr, i64, array<2 x  
i64>, array<2 x i64>)>  
    %18 = llvm.mlir.constant(10 : index) : i64  
    %19 = llvm.insertvalue %18, %17[3, 0] : !llvm.struct<(ptr, ptr, i64, array<2 x  
i64>, array<2 x i64>)>  
    %20 = llvm.mlir.constant(10 : index) : i64  
    %21 = llvm.insertvalue %20, %19[4, 0] : !llvm.struct<(ptr, ptr, i64, array<2 x  
i64>, array<2 x i64>)>  
  }
```



```

    %22 = llvm.mlir.constant(10 : index) : i64
    %23 = llvm.insertvalue %22, %21[3, 1] : !llvm.struct<(ptr, ptr, i64, array<2 x i64>, array<2 x i64>)>
    %24 = llvm.mlir.constant(1 : index) : i64
    %25 = llvm.insertvalue %24, %23[4, 1] : !llvm.struct<(ptr, ptr, i64, array<2 x i64>, array<2 x i64>)>
    %26 = llvm.mlir.constant(1 : index) : i64
    %27 = llvm.mlir.constant(0 : index) : i64
    %28 = llvm.mlir.constant(10 : index) : i64
    %29 = llvm.extractvalue %25[1] : !llvm.struct<(ptr, ptr, i64, array<2 x i64>, array<2 x i64>)>
    %30 = llvm.extractvalue %12[1] : !llvm.struct<(ptr, ptr, i64, array<2 x i64>, array<2 x i64>)>
    gpu.launch_func @square_kernel::@square_kernel blocks in (%28, %26, %26)
    threads in (%28, %26, %26) : i64 args(%27 : i64, %27 : i64, %29 : !llvm.ptr, %30 : !llvm.ptr)
    %31 = llvm.extractvalue %12[0] : !llvm.struct<(ptr, ptr, i64, array<2 x i64>, array<2 x i64>)>
    llvm.return %31 : !llvm.ptr
}

gpu.module @square_kernel [#nvvm.target<0 = 3, chip = "sm_90", features = "+ptx80">] {
    llvm.func @square_kernel(%arg0: i64, %arg1: i64, %arg2: !llvm.ptr, %arg3: !llvm.ptr) attributes {gpu.kernel, nvvm.kernel} {
        %0 = llvm.mlir.constant(10 : index) : i64
        %1 = nvvm.read.ptx.sreg.ctaid.x : i32
        %2 = llvm.sext %1 : i32 to i64
        %3 = nvvm.read.ptx.sreg.tid.x : i32
        %4 = llvm.sext %3 : i32 to i64
        %5 = llvm.add %arg0, %2 : i64
        %6 = llvm.add %arg1, %4 : i64
        %7 = llvm.mul %5, %0 : i64
        %8 = llvm.add %7, %6 : i64
        %9 = llvm.getelementptr %arg2[%8] : (!llvm.ptr, i64) -> !llvm.ptr, f32
        %10 = llvm.load %9 : !llvm.ptr -> f32
        %11 = llvm.fmul %10, %10 : f32
        %12 = llvm.mul %5, %0 : i64
        %13 = llvm.add %12, %6 : i64
        %14 = llvm.getelementptr %arg3[%13] : (!llvm.ptr, i64) -> !llvm.ptr, f32
        llvm.store %11, %14 : f32, !llvm.ptr
        llvm.return
    }
}

```

We can extract the GPU function from the module.

```
module {
  llvm.func @square_kernel(%arg0: i64, %arg1: i64, %arg2: !llvm.ptr, %arg3: !
llvm.ptr) attributes {gpu.kernel, nvvm.kernel} {
  %0 = llvm.mlir.constant(10 : index) : i64
  %1 = nvvm.read.ptx.sreg.ctaid.x : i32
  %2 = llvm.sext %1 : i32 to i64
  %3 = nvvm.read.ptx.sreg.tid.x : i32
  %4 = llvm.sext %3 : i32 to i64
  %5 = llvm.add %arg0, %2 : i64
  %6 = llvm.add %arg1, %4 : i64
  %7 = llvm.mul %5, %0 : i64
  %8 = llvm.add %7, %6 : i64
  %9 = llvm.getelementptr %arg2[%8] : (!llvm.ptr, i64) -> !llvm.ptr, f32
  %10 = llvm.load %9 : !llvm.ptr -> f32
  %11 = llvm.fmul %10, %10 : f32
  %12 = llvm.mul %5, %0 : i64
  %13 = llvm.add %12, %6 : i64
  %14 = llvm.getelementptr %arg3[%13] : (!llvm.ptr, i64) -> !llvm.ptr, f32
  llvm.store %11, %14 : f32, !llvm.ptr
  llvm.return
}
```

Then translate the MLIR module to LLVM IR.

```
; ModuleID = 'LLVMDialectModule'
source_filename = "LLVMDialectModule"

define ptx_kernel void @square_kernel(i64 %0, i64 %1, ptr %2, ptr %3) {
  %5 = call i32 @llvm.nvvm.read.ptx.sreg.ctaid.x()
  %6 = sext i32 %5 to i64
  %7 = call i32 @llvm.nvvm.read.ptx.sreg.tid.x()
  %8 = sext i32 %7 to i64
  %9 = add i64 %0, %6
  %10 = add i64 %1, %8
  %11 = mul i64 %9, 10
  %12 = add i64 %11, %10
  %13 = getelementptr float, ptr %2, i64 %12
  %14 = load float, ptr %13, align 4
  %15 = fmul float %14, %14
  %16 = mul i64 %9, 10
  %17 = add i64 %16, %10
```

```

%18 = getelementptr float, ptr %3, i64 %17
store float %15, ptr %18, align 4
ret void
}

; Function Attrs: nocallback nofree nosync nounwind speculatable willreturn
memory(none)
declare noundef i32 @llvm.nvvm.read.ptx.sreg.ctaid.x() #0

; Function Attrs: nocallback nofree nosync nounwind speculatable willreturn
memory(none)
declare noundef i32 @llvm.nvvm.read.ptx.sreg.tid.x() #0

attributes #0 = { nocallback nofree nosync nounwind speculatable willreturn
memory(none) }

!llvm.module.flags = !{!0}

!0 = !{i32 2, !"Debug Info Version", i32 3}

```

Then compile to PTX using the nvptx backend to LLVM, which we invoke via `llc`.

```

//
// Generated by LLVM NVPTX Back-End
//

.version 7.8
.target sm_90
.address_size 64

        // .globl      square_kernel      // -- Begin function square_kernel
                                           // @square_kernel
.visible .entry square_kernel(
    .param .u64 square_kernel_param_0,
    .param .u64 square_kernel_param_1,
    .param .u64 .ptr .align 1 square_kernel_param_2,
    .param .u64 .ptr .align 1 square_kernel_param_3
)
{
    .reg .b32    %r<3>;
    .reg .f32    %f<3>;
    .reg .b64    %rd<15>;

// %bb.0:
    ld.param.u64    %rd1, [square_kernel_param_0];

```

```

ld.param.u64    %rd2, [square_kernel_param_3];
cvta.to.global.u64    %rd3, %rd2;
ld.param.u64    %rd4, [square_kernel_param_1];
ld.param.u64    %rd5, [square_kernel_param_2];
cvta.to.global.u64    %rd6, %rd5;
mov.u32         %r1, %ctaid.x;
cvt.s64.s32     %rd7, %r1;
mov.u32         %r2, %tid.x;
cvt.s64.s32     %rd8, %r2;
add.s64         %rd9, %rd1, %rd7;
add.s64         %rd10, %rd4, %rd8;
mad.lo.s64      %rd11, %rd9, 10, %rd10;
shl.b64         %rd12, %rd11, 2;
add.s64         %rd13, %rd6, %rd12;
ld.global.f32   %f1, [%rd13];
mul.rn.f32      %f2, %f1, %f1;
add.s64         %rd14, %rd3, %rd12;
st.global.f32   [%rd14], %f2;
ret;

                                // -- End function
}

```

MLIR Pipeline

Now we can construct two modules `compile.py` and `run.py` module implements our GPU code generation pipeline, transforming high-level MLIR into optimized PTX assembly.

The full source code is available [on Github](#) and also as a [notebook](#).

In our example we define a primary function `compile_mlir_to_ptx` that orchestrates the entire compilation process. Inside, it first parses the MLIR string into a module representation, then applies our GPU compilation pipeline through a carefully ordered sequence of transformation passes. These passes progressively lower the code from high-level tensor operations to GPU-specific constructs, including bufferization to handle memory accesses, converting linalg operations to affine loops, mapping those loops to GPU blocks and threads, and finally generating NVIDIA-specific code via the NVVM dialect. After transformations, the function extracts the GPU module and converts it to PTX assembly using LLVM tools, resulting in code that can be directly executed on NVIDIA GPUs without further compilation.

```

import subprocess

from mlir.ir import Context, Module
from mlir.passmanager import PassManager

def compile_mlir_to_ptx(mlir_module_str: str, chip_type="sm_75"):
    """Compiles MLIR module string to PTX code."""

```

```

with Context():
    # Parse the input module
    module = Module.parse(mlir_module_str)

    # Apply GPU compilation pipeline
    module, gpu_module = apply_gpu_pipeline(module, chip_type)

    # Generate PTX from the GPU module
    ptx = generate_ptx(str(gpu_module), chip_type)

    return ptx

def apply_gpu_pipeline(module, chip_type="sm_75"):
    """Applies the GPU compilation pipeline to the MLIR module."""
    pm = PassManager()
    pm.enable_ir_printing(print_after_change=True)
    pm.add("canonicalize")
    pm.add(
        "one-shot-bufferize{ bufferize-function-boundaries function-boundary-type-
conversion=identity-layout-map }"
    )
    pm.add("canonicalize")
    pm.add("convert-linalg-to-affine-loops")
    pm.add("func.func(affine-loop-invariant-code-motion)")
    pm.add("func.func(convert-affine-for-to-gpu)")
    pm.add("gpu-kernel-outlining")
    pm.add("lower-affine")
    pm.add("gpu-decompose-memrefs")
    pm.add("expand-strided-metadata")
    pm.add("normalize-memrefs")
    pm.add(
        "gpu.module(convert-gpu-to-nvvm{index-bitwidth=0 use-bare-ptr-memref-call-
conv })"
    )
    pm.add(f"nvvm-attach-target{ {chip={chip_type} features=+ptx80 0=3} }")
    pm.add("convert-nvvm-to-llvm")
    pm.add("reconcile-unrealized-casts")
    pm.add("gpu-to-llvm { use-bare-pointers-for-host use-bare-pointers-for-kernels
}")
    pm.run(module.operation)

    # Extract the GPU module

```

```

gpu_module = extract_gpu_module(module)

return module, gpu_module

def extract_gpu_module(module: Module) -> Module:
    """Extracts the GPU module from a transformed MLIR module."""
    # Navigate the operation tree to find the GPU module
    # Structure: module -> region[0] -> block[0] -> operations[1] (GPU host-device
code)
    # -> region[0] -> block[0] -> operations[0] (GPU module)
    try:
        main_func_op = module.operation.regions[0].blocks[0].operations[1]
        gpu_module_op = main_func_op.regions[0].blocks[0].operations[0]

        # Create a new module from the GPU module operation
        gpu_module = Module.parse(str(gpu_module_op))
        return gpu_module
    except (IndexError, AttributeError) as e:
        raise RuntimeError(f"Failed to extract GPU module: {e}") from e

def generate_ptx(gpu_module_str, chip_type="sm_75"):
    """Generates PTX from an MLIR GPU module string."""
    # First convert MLIR to LLVM IR
    llvm_ir_result = subprocess.run(
        ["mlir-translate", "--mlir-to-llvmir", "-"],
        input=gpu_module_str,
        capture_output=True,
        text=True,
    )

    if llvm_ir_result.returncode != 0:
        print("Error generating LLVM IR:")
        print(llvm_ir_result.stderr)
        return None

    llvm_ir = llvm_ir_result.stdout

    # Then convert LLVM IR to PTX
    ptx_result = subprocess.run(
        ["llc", "-march=nvptx64", f"-mcpu={chip_type}", "-"],
        input=llvm_ir,

```

```

        capture_output=True,
        text=True,
    )

    if ptx_result.returncode != 0:
        print("Error generating PTX:")
        print(ptx_result.stderr)
        return None

    return ptx_result.stdout

```

Now we can define the `run_kernel` function which serves as the interface between our Python code and the GPU hardware, managing the execution of our compiled PTX kernels. It takes a PTX code string, kernel name, arguments with their types, and the grid and block dimensions that define our parallelization strategy. The function first loads the PTX code into a CUDA module using the CUDA driver API, then retrieves a handle to the named kernel function. After preparing the kernel arguments in the format expected by the CUDA runtime, it launches the kernel with the specified execution configuration, determining how many thread blocks and threads per block will process our data.

```

def run_kernel(
    ptx_code,
    kernel_name,
    args,
    arg_types,
    grid_dims,
    block_dims,
):
    """Run a PTX kernel."""
    module = checkCudaErrors(cu.cuModuleLoadData(ptx_code.encode("utf-8")))

    kernel_func = checkCudaErrors(
        cu.cuModuleGetFunction(module, kernel_name.encode("utf-8"))
    )

    kernel_args = (tuple(args), tuple(arg_types))

    checkCudaErrors(
        cu.cuLaunchKernel(
            kernel_func,
            grid_dims[0],
            grid_dims[1],
            grid_dims[2],
            block_dims[0],
            block_dims[1],
            block_dims[2],

```



```

        0, # shared memory bytes
        0, # stream
        kernel_args, # kernel args
        0, # extra
    )
)

checkCudaErrors(cu.cuCtxSynchronize())

checkCudaErrors(cu.cuModuleUnload(module))

```

The following code is a complete end-to-end example of GPU kernel execution using our compilation pipeline. It first compiles MLIR code from a file into PTX assembly, then sets up a CUDA context and allocates memory on both host and device. The example prepares a square matrix operation where each thread processes one element, organizing computation with a grid dimension matching the matrix rows and block dimension matching the columns. After copying input data to the GPU, it executes the kernel with the appropriate arguments and dimensions, then retrieves the results back to the host into a NumPy array.

```

import ctypes
import numpy as np
import cuda.cuda as cu
import cuda.cudart as cudart
import cuda.nvrtc as nvrtc
import subprocess
from compile import compile_mlir_to_ptx
from run import run_kernel

ptx_code = compile_mlir_to_ptx(open('square.mlir').read())

cuda_context = setup_cuda()

try:
    # Allocate device memory
    d_input = allocate_device_memory(input_data.nbytes)
    output_data = np.zeros((size, size), dtype=np.float32)
    d_output = allocate_device_memory(output_data.nbytes)

    # Copy input data to device
    copy_host_to_device(input_data, d_input)

    # Run kernel
    grid_dims = (size, 1, 1) # One thread block per row
    block_dims = (size, 1, 1) # One thread per column

    # Prepare arguments according to the PTX code

```

```

# square_kernel(
#     .param .u64 square_kernel_param_0,           // Grid dimension
offset
#     .param .u64 square_kernel_param_1,           // Block dimension
offset
#     .param .u64 .ptr .align 1 square_kernel_param_2, // Input pointer
#     .param .u64 .ptr .align 1 square_kernel_param_3 // Output pointer
# )
args = [
    0, # Grid dimension offset
    0, # Block dimension offset
    d_input, # Input pointer
    d_output, # Output pointer
]
arg_types = [ctypes.c_int, ctypes.c_int, None, None] # Using None for pointer
types

print("Running kernel on GPU...")
run_kernel(
    ptx_code,
    "square_kernel",
    args,
    arg_types,
    grid_dims,
    block_dims,
)

# Copy results back to host
copy_device_to_host(d_output, output_data)

# Verify results
print("Verifying results...")
np.testing.assert_allclose(output_data, expected_output, rtol=1e-5)
print("Success! Results verified.")

finally:
    # Clean up resources
    free_device_memory(d_input)
    free_device_memory(d_output)
    cleanup_cuda(cuda_context)

```

And that's it, it's the skeleton of a very tiny compiler pipeline which takes dynamic tensor expressions in MLIR and then compiles them to PTX and executes them on the GPU. Now our goal moving forward is to target that same MLIR from something more natural to program in like a eDSL in Python to express kernel operations and

then have our pipeline take that and compile it to MLIR and then for the GPU. More on this in the next section.

Embedding Binaries

As an aside, the alternative path to compiling a GPU kernel is to compile it to a binary and then embed that binary in the MLIR module and then have the host code load the binary and launch the kernel.

The `gpu.binary` operation in MLIR represents a compiled GPU kernel that can be loaded and executed on a GPU device. It encapsulates the binary representation of GPU code, which can be in various formats such as PTX assembly, CUBIN binary, or HSACO for AMD GPUs. This operation is particularly useful when you want to embed pre-compiled GPU kernels directly into your MLIR module, allowing for direct execution without going through the compilation pipeline at runtime.

The `gpu-module-to-binary` pass is a transformation that converts GPU modules into GPU binaries. This pass scans through the MLIR module to find all nested GPU modules and serializes them according to the target attributes attached to each module. It produces a GPU binary with an object for every target architecture specified. The pass supports various output formats including offloading representation, assembly code, binaries, and fatbinaries.

All this is doing under the hood is running `ptxas` on the PTX code and embedding the output in the MLIR module with the given arguments the `gpu-module-to-binary` pass passed to the `ptxas` command. For example the following MLIR module contains a GPU binary for the `sm_70` architecture.

```
module attributes {gpu.container_module} {
  gpu.binary @vecadd_kernel [
    #gpu.object<#nvvm.target<chip = "sm_70">,
    offload = "BC\C0\DE5\14 [... truncated ...] kernel20.1.0nvptx64-nvidia-
cudaLLVMDialectModule\00\00\00\00">]
}
```

Then in the host code you can launch the kernel with the following:

```
gpu.launch_func @vecadd_kernel::@vecadd_kernel
  blocks in (%0, %0, %0)
  threads in (%0, %0, %0) : i64
  args(%1 : i32, %2 : i64)
```

External Resources

[CUDA Installation Guide for Linux](#)

[CUDA C Programming Guide](#)

[CUDA C Best Practices Guide](#)

[CUDA Toolkit Documentation 12.6](#)

[CUDA Python API](#)

[CUDA Python launchKernel](#)

[CUDA Refresher: The CUDA Programming Model](#)

[NVVM IR Specification](#)

[LLVM User Guide for NVPTX Back-end](#)

[Cuda Quick Start Guide](#)

[2024 EuroLLVM - Zero to Hero: Programming Nvidia Hopper Tensor Core with MLIR's NVGPU Dialect](#)

[A guide to Dockerfiles for building LLVM](#)

[Understanding PTX: The Assembly Language of CUDA GPU Computing](#)

[cuBLAS \(nvmath.bindings.cublas\)](#)

[cuda.core - Pythonic access to CUDA Runtime](#)